

NDgpu — tri-S_N: what CUDA graph capture really costs (Colab)

The companion to `colab_gpu_fusion_phase1.ipynb`, which does not touch `TriSNTransportSolver` at all. Everything measured there is diffusion/SPN; this notebook covers the transport sweep, and specifically a finding that the other notebook cannot see.

The question. Does graph capture help or hurt large problems?

The short answer. Replay is never slower — a captured graph runs the same kernels, and one `cudaGraphLaunch` replaces N driver-side launches. But capture *forbids* cuBLAS and forbids allocation inside the captured region, and `tri_sn`'s level sweep contains three weighted contractions: the step-scheme flux reduction, the SCB flux reduction, and the per-cell 3×3 corner matvec. To stay capturable, all three had been written as a broadcast-multiply into a full-size temporary followed by a reduction — the source comments said so outright (*"NOT matmul -- cuBLAS calls cannot be captured into a CUDA graph"*).

That form does **~3× the memory traffic** of the contraction it computes, on every level of every sweep, and it holds scratch as large as the angular flux itself. So capture was paying in **bandwidth and footprint** — what dominates large problems — to buy **launch overhead**, which only matters on small ones. And it was unconditional.

The fix, and what this notebook tests. Not gating or removing capture, but removing the deoptimization: all three contractions are now single hand-written kernels (`kernels.moment_gather`, `kernels.batched_matvec`) that are *still* capturable, because only cuBLAS is unrecordable — an `ElementwiseKernel` records fine. So large problems get matmul-equivalent traffic **and** small ones keep graph replay.

section	measures
1	is capture actually working on this GPU (and why not, if not)
2	correctness gate: the two formulations must agree exactly
3	sweep buffer footprint, both ways (exact allocation accounting)
4	wall time vs problem size — the claim that the win <i>grows</i>
5	capture on/off, which answers the original question directly
6	the 3D prism case, where footprint binds hardest
7	Phases 1b/2/3/6 on the transport path (hybrid + 11-group tri-S_N)

Two switches drive it. `kernels.set_fused_group("block", ...)` selects the contraction formulation and **must be set before the solver is constructed** — the flag is latched at setup, since a captured graph records whichever form was in force and buffers are sized from it. `graphs=` on the solver turns capture itself on and off.

```
import os
try:
    # Colab: upload dist/ndgpu-src.z
    from google.colab import files
    uploaded = files.upload()
    zip_name = next(iter(uploaded))
    get_ipython().run_line_magic("pip", f"install -q {zip_name}")
except ImportError:
    import cupy
    get_ipython().run_line_magic("pip", "install -q cupy-cuda12x")
    get_ipython().system("nvidia-smi -L")
except ImportError:
    # local run: ndgpu already imported
    pass
```

[Browse...](#) ndgpu-src.zip

```
ndgpu-src.zip(application/x-zip-compressed) - 296009 bytes, last modified: n/a - 100% done
Saving ndgpu-src.zip to ndgpu-src.zip
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheel for ndgpu (pyproject.toml) ... done
GPU 0: Tesla T4 (UUID: GPU-6ca6f0ce-8dcf-e0ee-2e05-244075c26b3c)
```

```
import os
import numpy as np

from ndgpu import device_name, get_backend, kernels, profiling
from ndgpu.benchmarks import build_hpmr2d, build_hpmr3d
from ndgpu.tri_sn import TriSNTTransportSolver

xp = get_backend("auto")
GPU = kernels.is_cupy(xp)
print("backend:", device_name(xp))
if not GPU:
    print("\n*** No GPU: capture is CuPy-only and every ratio below will be
        ~1.0. Switch the Colab runtime to a GPU. ***")

QUICK = bool(os.environ.get("NDGPU_QUICK"))
TOL = dict(tol_k=1e-7, tol_source=1e-6)

def solver(prob, scheme="step", fused=True, graphs=True, n_azi=12, **kw):
    """Build a tri-S_N solver with the contraction form latched at setup.
```

```

set_fused_group must bracket the *constructor*, not the solve: the buffers
are sized from the flag and a captured graph records the form in force.
"""
prev = kernels.set_fused_group("block", fused)
try:
    s = TriSNTransportSolver(
        prob.grid, prob.materials, prob.material_map, active=prob.active
        bc="vacuum", scheme=scheme, engine="levels" if GPU else "levels"
        device="auto", n_polar=2, n_azi=n_azi,
        mix_material=prob.mix_material, mix_weight=prob.mix_weight,
        graphs=graphs, **kw)
finally:
    kernels.set_fused_group("block", prev)
assert s._fused_reduce == (fused and GPU), (s._fused_reduce, fused, GPU)
return s

def buffer_bytes(s):
    """Total sweep-buffer footprint, including the per-level work tuples."""
    b = sum(v.nbytes for v in s._bufs.values() if hasattr(v, "nbytes"))
    return b + sum(x.nbytes for t in s._bufs["work"] for x in t)

hp = build_hpnr2d(refine=3 if QUICK else 4, drum_angle_deg=0.0, absorber="po
print(f"\nHP-MR 2D: {hp.grid.n_cells} cells")

backend: cuda (cupy): Tesla T4

HP-MR 2D: 11250 cells

```

✓ 1. Is capture actually working here?

`graphs_active` is `None` until a GPU sweep has run, then `True` if the level loop was recorded or `False` if capture failed — in which case `_graph_error` carries the reason and the solver has permanently fallen back to the plain loop. Worth checking first: every number below is meaningless if capture silently never engaged.

```

s = solver(hp, scheme="step", fused=True, graphs=True)
r = s.solve(**TOL)
print(f"k_eff = {r.k_eff:.7f}  ({r.outer_iterations} outers, {r.n_sweeps} sw
print("graphs_active:", s.graphs_active)
if s._graph_error:
    print("capture fell back:", s._graph_error)
elif s.graphs_active:
    print(f"captured {len(s._graphs)} graphs (one per (group, iface) pair)")

k_eff = 0.9883849  (10 outers, 265 sweeps)
graphs_active: True
captured 2 graphs (one per (group, iface) pair)

```

✓ 2. Correctness gate

The two formulations compute the same sums in a different association, so a converged solve must agree to round-off. The sweep count must match too — that checks the *iteration path* is identical, not merely the final answer. Both schemes, because they use different contractions: `step` has one flux reduction, `scb` has a flux reduction *and* the per-cell 3×3 matvec inside the level loop.

This is the assertion that matters most in the whole notebook: it is the first time the fused contractions run as real CUDA kernels inside a captured graph.

```
gate = []
for scheme in ("step", "scb"):
    ref = solver(hp, scheme=scheme, fused=False).solve(**TOL)
    got = solver(hp, scheme=scheme, fused=True).solve(**TOL)
    assert ref.converged and got.converged
    dk = abs(got.k_eff - ref.k_eff)
    flux_err = float(np.abs(np.asarray(got.flux) - np.asarray(ref.flux)).max()
                     / max(np.abs(np.asarray(ref.flux)).max(), 1e-300))
    gate.append(dict(scheme=scheme, k_eff=f"{got.k_eff:.7f}", dk=f"{dk:.1e}"
                    flux_relerr=f"{flux_err:.1e}",
                    sweeps=f"{got.n_sweeps} vs {ref.n_sweeps}"))
    assert dk < 1e-9, f"{scheme}: fused contractions moved k by {dk:.2e}"
    assert flux_err < 1e-7, f"{scheme}: flux moved by {flux_err:.2e}"
    assert got.n_sweeps == ref.n_sweeps, f"{scheme}: iteration path changed"
    profiling.report(gate, ["scheme", "k_eff", "dk", "flux_relerr", "sweeps"],
                    "broadcast vs fused contractions")
print("\ngate passed")
```

```
=== broadcast vs fused contractions ===
scheme      k_eff      dk flux_relerr      sweeps
step  0.9883849  4.4e-16    6.4e-15  265 vs 265
scb    0.9937038  1.1e-16    3.7e-15  118 vs 118

gate passed
```

✓ 3. Sweep buffer footprint

Exact allocation accounting, not a timing — this number is certain. The buffers that disappear are the intermediates of the three contractions: `psiw` is (M, N) , the size of the angular flux itself, and the SCB per-level `bm` blocks total three times it. On the fused path they are pure footprint, and footprint is what caps the largest 3D cases.

```

rows = []
for scheme in ("step", "scb"):
    a = buffer_bytes(solver(hp, scheme=scheme, fused=False))
    f = buffer_bytes(solver(hp, scheme=scheme, fused=True))
    rows.append(dict(scheme=scheme, broadcast_MB=a / 1e6, fused_MB=f / 1e6,
                    saved_pct=100 * (a - f) / a))
profiling.report(rows, ["scheme", "broadcast_MB", "fused_MB", "saved_pct"],
                  f"sweep buffers, HP-MR 2D ({hp.grid.n_cells} cells)")
if not GPU:
    print("\n(both columns match on CPU: the fused contractions are GPU-only
          "\n _fused_reduce is False in both legs and no buffer is skipped.
          "\n GPU the fused column drops -- ~23% (step) / ~30% (scb) at this
else:
    print("\n(scales linearly with cells x ordinates)")

=== sweep buffers, HP-MR 2D (11250 cells) ===
scheme broadcast_MB fused_MB saved_pct
step      9.569      7.409      22.57
scb       33.63      23.49      30.15

(scales linearly with cells x ordinates)

```

✓ 4. Wall time vs problem size

The claim under test: because the penalty was memory traffic, the fused form should win *more* as the problem grows — the opposite of how a launch-overhead optimization behaves. If the ratio is flat and ~1.0, the traffic argument is wrong and the broadcast form was never actually costing anything.

Ordinate count is a second axis on the same effect: the contractions are over the M ordinates, so raising `n_azi` grows the temporaries without touching the mesh.

```

def timed(prob, scheme, fused, n_azi, n=1 if QUICK else 3):
    best, res = float("inf"), None
    for _ in range(n):
        s = solver(prob, scheme=scheme, fused=fused, n_azi=n_azi)
        r = s.solve(**TOL)
        assert r.converged, r
        best, res = min(best, r.solve_seconds), r
    return res, best

cases = ([ (4, 12) ] if QUICK else [ (3, 12), (4, 12), (4, 24), (6, 12) ])
rows = []
for refine, n_azi in cases:
    prob = build_hpmr2d(refine=refine, drum_angle_deg=0.0, absorber="polar")
    for scheme in ("step", "scb"):

```

```

r_a, t_a = timed(prob, scheme, False, n_azi)
r_f, t_f = timed(prob, scheme, True, n_azi)
assert abs(r_f.k_eff - r_a.k_eff) < 1e-9
rows.append(dict(scheme=scheme, cells=prob.grid.n_cells, n_azi=n_azi
                 M=r_f.n_ordinates, k_eff=f"{r_f.k_eff:.7f}",
                 broadcast_s=t_a, fused_s=t_f, speedup=t_a / t_f))
profiling.report(rows, ["scheme", "cells", "n_azi", "M", "k_eff",
                        "broadcast_s", "fused_s", "speedup"],
                 f"tri-S_N solve, best of {1 if QUICK else 3}")

```

```

=== tri-S_N solve, best of 3 ===

```

scheme	cells	n_azi	M	k_eff	broadcast_s	fused_s	speedup
step	6498	12	24	0.9839867	2.357	2.362	0.9981
scb	6498	12	24	0.9899604	3.873	2.875	1.347
step	11250	12	24	0.9883849	5.052	4.973	1.016
scb	11250	12	24	0.9937038	4.263	3.458	1.233
step	11250	24	48	0.9886236	4.972	4.952	1.004
scb	11250	24	48	0.9938797	6.265	4.79	1.308
step	24642	12	24	0.9934476	4.12	4.227	0.9747
scb	24642	12	24	0.9976670	6.287	4.807	1.308

✓ 5. Capture on vs off — the original question, directly

With the deoptimization gone, capture should be a pure win again: a clear gain on the small mesh, shrinking toward neutral as the kernels get long enough to hide their launch cost. What it must *not* be is negative — if it is, replay is not the whole story on this GPU and the capture path deserves a size gate after all.

Run with the fused contractions in both legs, so this isolates capture itself rather than re-measuring section 4.

```

rows = []
for refine in ([4] if QUICK else [3, 4, 6]):
    prob = build_hpmr2d(refine=refine, drum_angle_deg=0.0, absorber="polar")
    out = {}
    for tag, use in (("graphs", True), ("no_graphs", False)):
        best, res, act = float("inf"), None, None
        for _ in range(1 if QUICK else 3):
            s = solver(prob, scheme="step", fused=True, graphs=use)
            r = s.solve(**TOL)
            assert r.converged, r
            best, res, act = min(best, r.solve_seconds), r, s.graphs_active
        out[tag] = (res, best, act)
    r_g, t_g, act = out["graphs"]
    r_n, t_n, _ = out["no_graphs"]
    assert abs(r_g.k_eff - r_n.k_eff) < 1e-12, "capture must not change the
rows.append(dict(cells=prob.grid.n_cells, sweeps=r_g.n_sweeps,

```

```

        captured=act, graphs_s=t_g, no_graphs_s=t_n,
        speedup=t_n / t_g))
profiling.report(rows, ["cells", "sweeps", "captured", "graphs_s",
                        "no_graphs_s", "speedup"],
                "CUDA graph capture on vs off (fused contractions both legs)

```

```

=== CUDA graph capture on vs off (fused contractions both legs) ===
cells  sweeps  captured  graphs_s  no_graphs_s  speedup
  6498    315      True    2.347      5.451      2.323
 11250    265      True    5.004      9.103      1.819
 24642    112      True    4.119      5.866      1.424

```

✓ 6. The 3D prism case

The largest configuration, and the one where footprint binds hardest: the 2D radial core extruded to 200 cm, so the angular flux and every contraction temporary grow by the axial layer count. Run last because it is much the most expensive cell here.

```

hp3 = build_hpmr3d(refine=3 if QUICK else 4, nz=10, # must be a multiple o
                  drum_angle_deg=0.0, absorber="polar")
print(f"HP-MR 3D: {hp3.grid.n_cells} cells")

rows = []
for scheme in ("step", "scb"):
    a = buffer_bytes(solver(hp3, scheme=scheme, fused=False))
    f = buffer_bytes(solver(hp3, scheme=scheme, fused=True))
    r_a, t_a = timed(hp3, scheme, False, 12, n=1 if QUICK else 2)
    r_f, t_f = timed(hp3, scheme, True, 12, n=1 if QUICK else 2)
    assert abs(r_f.k_eff - r_a.k_eff) < 1e-9
    rows.append(dict(scheme=scheme, cells=hp3.grid.n_cells,
                    k_eff=f"{r_f.k_eff:.7f}",
                    broadcast_MB=a / 1e6, fused_MB=f / 1e6,
                    broadcast_s=t_a, fused_s=t_f, speedup=t_a / t_f))
profiling.report(rows, ["scheme", "cells", "k_eff", "broadcast_MB", "fused_M
                        "HP-MR 3D prisms: footprint")
profiling.report(rows, ["scheme", "broadcast_s", "fused_s", "speedup"],
                "HP-MR 3D prisms: solve time")

```

HP-MR 3D: 112500 cells

```

=== HP-MR 3D prisms: footprint ===
scheme  cells      k_eff  broadcast_MB  fused_MB
  step  112500  0.9402631      116      94.36
  scb   112500  0.9454970     397.1     295.7

```

```

=== HP-MR 3D prisms: solve time ===
scheme  broadcast_s  fused_s  speedup
  step      153.9    153.9      1

```


scb	357.9	258.3	1.386
-----	-------	-------	-------

✓ 7. The other recent phases, on this path

Phases 1b (fused triangular stencil), 2 (order-N blocks), 3 (Krylov kernels) and 6 (batched multigroup source) were measured in the companion notebook on *diffusion and SPN*. This section checks them where they land on the transport pipeline: the **hybrid tri-S_N / diffusion** solver, which runs S_N in the drum cells and tri diffusion in the bulk, and is the HP-MR configuration the drum-worth work actually uses.

Four legs as in the companion notebook, so a win is attributable to one kernel family rather than to "fusion".

One honest caveat, and the notebook is set up to show it rather than hide it: **Phase 6 did not touch tri_sn's own group loop.** `TriSNTTransportSolver.solve` assembles its in-scatter with its own $O(G^2)$ Python loop (twice — once for the group solves and once for the CMFD pass), independent of the power iteration in `solver.py` that Phase 6 batched. So on a pure tri-S_N solve `x_groups` measures only the diffusion/CMFD sub-solves, and the 11-group row below is the evidence for whether extending Phase 6 there is worth doing.

```
from ndgpu.benchmarks.hpmr import hpmr_materials_builtin, hpmr_transport_mask
from ndgpu.benchmarks.hpmr_assembly import build_hpmr_assembly2d
from ndgpu.hybrid_tri_sn import HybridTriSNDiffusionSolver

OFF = dict(stencil=False, krylov=False, block=False, groups=False)
LEGS = [("none", OFF),
        ("stencil", {**OFF, "stencil": True, "block": True}),
        ("krylov", {**OFF, "krylov": True}),
        ("groups", {**OFF, "groups": True}),
        ("all", {k: True for k in OFF})]

def ab_legs(make, label, n=1 if QUICK else 3):
    """Best-of-n per leg; k must agree across legs to 2e-8."""
    t, k = {}, {}
    for tag, groups in LEGS:
        prev = {g: kernels.set_fused_group(g, f) for g, f in groups.items()}
        try:
            make().solve(tol_k=1e-3, tol_source=1e-2)          # warm-up
            best, res = float("inf"), None
            for _ in range(n):
                r = make().solve(**TOL)
                assert r.converged, r
                best, res = min(best, r.solve_seconds), r
            t[tag], k[tag] = best, res
```

```

        t[tag], k[tag] = best, res.k_eff
    finally:
        for g, f in prev.items():
            kernels.set_fused_group(g, f)
        spread = max(k.values()) - min(k.values())
        assert spread <= 2e-8, f"{label}: fusion moved k by {spread:.2e}"
        row = dict(case=label, k_eff=f"{k['all']:.7f}")
        row.update({f"x_{tag}": t["none"] / t[tag] for tag, _ in LEGS[1:]})
        row["none_s"], row["all_s"] = t["none"], t["all"]
    return row

rows = []

# Hybrid S_N-in-drums / diffusion-in-bulk: the HP-MR drum-worth configuration.
mask = hpmr_transport_mask(hp, region="drum")
hyb_kw = dict(active=hp.active, sn_mask=mask, mask_bc=hp.mask_bc,
              device="auto", n_polar=2, n_azi=12,
              mix_material=hp.mix_material, mix_weight=hp.mix_weight)
rows.append(ab_legs(
    lambda: HybridTriSNDiffusionSolver(hp.grid, hp.materials, hp.material_map,
                                         **hyb_kw),
    f"hybrid tri-SN/diffusion ({hp.grid.n_cells} cells, 2 group)"))

# Pure tri-S_N on the real 11-group data: the row that says whether extending
# Phase 6 into tri_sn's own group loop is worth it.
g11 = hpmr_materials_builtin(build_hpmr_assembly2d(refine=3).materials[1])
hp11 = build_hpmr2d(refine=3 if QUICK else 4, drum_angle_deg=0.0,
                    absorber="polar", materials=g11)
rows.append(ab_legs(
    lambda: solver(hp11, scheme="step", fused=True),
    f"tri-SN, 11 group ({hp11.grid.n_cells} cells)"))

profiling.report(rows, ["case", "k_eff", "none_s", "all_s"],
                  "recent phases on the transport path: cost")
profiling.report(rows, ["case", "x_stencil", "x_krylov", "x_groups", "x_all"],
                  "speedup vs no fusion (>1 = faster)")

```

```
=== recent phases on the transport path: cost ===
```

	case	k_eff	none_s	all_s
hybrid tri-SN/diffusion (11250 cells, 2 group)		1.0009374	12.65	9.838
tri-SN, 11 group (11250 cells)		0.8878791	30.23	20.15

```
=== speedup vs no fusion (>1 = faster) ===
```

	case	x_stencil	x_krylov	x_groups
hybrid tri-SN/diffusion (11250 cells, 2 group)		1.038	1.076	0.9703
tri-SN, 11 group (11250 cells)		1.445	1.02	1.002

8. What the answer looks like

- **Section 3 is certain** — it is allocation accounting. On CPU it already reads 23% (step) and 30% (SCB) of the sweep buffers removed at HP-MR 2D refine 4, and it scales with cells $\times$ ordinates.
- **Section 4 is the claim that needed a GPU.** A speedup that *grows* with cells and with `n_azi` confirms the penalty was memory traffic. Flat and ~ 1.0 would mean the broadcast form was never costing anything and the rework bought only the footprint.
- **Section 5 answers the original question.** Expect a clear win on the small mesh decaying toward 1.0 — capture removes launch overhead and nothing else. A ratio below 1.0 anywhere would be the surprise, and would justify a size gate on capture that the current code deliberately does not have.

If sections 4 and 5 both come out flat, the honest conclusion is that this rework bought footprint and code clarity but not time — worth recording either way, and cheaper to know than to assume.

Untouched by all of this: the hex stencil (`hex.py`), Chronopoulos-Gear pipelined CG, extending capture beyond `tri_sn`, and float32/mixed precision for the `S_N` sweeps (which have no `dtype` parameter at all).